ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2

DFSha

Proyecto 1: *Diseño e implementación de un sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad (DFSha)*

Objetivo General:

Diseñar e implementar un **DFS** que permita almacenar y acceder a archivos grandes de forma distribuida en múltiples nodos, considerando alta disponibilidad, rendimiento, transparencia y seguridad. Este DFS debe permitir que un cliente cargue y recupere archivos (tanto la escritura como la lectura deben ser distribuidas entre varios nodos).

Descripción del servicio

Típicamente estos sistemas pueden ser modelados e implementados con al menos (generalmente existen más) estas componentes:

Cliente DFSha: Programa que es usuario del sistema dfsha y que con una abstracción de sistema de archivos jerárquico tipo Linux, puede realizar operaciones de:

- RF1: Gestión del sistema de archivos: ls, cd, mkdir, rmdir, rm, etc.
- RF2: Transferencia de archivos: send() y receive()
- RF3: Acceso a archivos: open(), close(), read(), write(), lock(), etc. Realmente las operaciones send() y recv() pueden ser implmentadas con estas primitivas, pero hay diferencias fundamentales entre ambos tipos de servicios de archivos: transferencia y acceso.

Normalmente, el programa Cliente no es parte del servicio, y desde ese punto de vista es implementado como una arquitectura cliente/servidor, sin embargo a continuación describiremos opciones de arquitectura para el lado del Servidor (servicio).

Servicio DFSha: El servicio puede ser diseñado e implementado siguiendo varias arquitecturas de sistemas de archivos distribuidos (dfs), igualmente e independiente de la arquitectura debe cumplir entre otros los siguientes requisitos:

- Escalabilidad en número de usuarios, nro de archivos, tamaño de archivos, etc.
- Alta disponibilidad, el servicio debe ser altamente redundante para evitar caídas del servicio o pérdida de archivos (datos).
- Como una consecuencia del punto anterior, estos sistemas enfrentan el reto de la consistencia de datos, problema que se debe resolver en este DFSha

- Particionamiento de archivos: Los archivos deben estar distribuidos entre varios nodos, y a su vez cada archivo debe estar distribuido entre varios nodos.
- Rendimiento, dada la alta escalabilidad deseada, el servicio debe soportar concurrencia en el acceso a los archivos y datos dentro de un archivo.
- Seguridad, el sistema debe garantizar encriptación de los datos, tanto en las comunicaciones como en el almacenamiento de los datos, además:
    - El servicio también debe garantizar control de acceso a los archivos / directorios por usuario o grupo desde aspectos tan básicos como user/password, ACL, etc, hasta 2FA, API Keys, etc.
    - También debe garantizar seguridad entre nodos de datos y control.
- Transparencia, el cliente debe tener diferentes niveles de transparencia de acceso y localización a los archivos del servicio DFSha, desde una transparencia 100% (el cliente cree que esta accediendo los archivos de forma local a su máquina) o relajando esta transparencia a mecanismos dinámicos de acceso a los archivos, pero nunca de forma estática.
- Puede aclararse o modificarse alguno de los requisitos anterios o venir nuevos dados por las siguientes especificaciones:
    - …
    - …
    - …

El servicio DFSha puede ser implementado utilizando una combinación de arquitecturas:

Opción 1: C/S, en el cual el S es implementado por mecanismos de Composición y Distribución del servicio (S2S). En este acercamiento se considera un sistema autónomo, es decir, el servicio es diseñado e implementado por una misma organización y define mecanismo abiertos para acceso al servicio entre C y S, pero mantiene la autonomía S-S. La implementación del servicio S corre dentro de una red privada real o virtual.

Opción 2: P2P, en el cual el servicio es implementado por un conjunto de SuperPeer, que pertenecen a organizaciones diferentes (sistema federado e integración de sistemas autónomos). Estos servicios así corren en una red superpuesta sobre Internet configurada a través de una red privada virtual.

Marco teórico:

Un sistema de archivos distribuidos, permite compartir y acceder de forma concurrente un conjunto de archivos que se encuentran almacenados en diferentes nodos. Uno de los sistemas más maduros, vigente y antiguo de estos sistemas es el NFS (Network File System) desarrollado en su momento por Sun Microsystems y que hoy en día es ampliamente usado en sistemas Linux. Hay otros sistemas de archivos distribuidos como AFS (Andrew File System) y SMB (Server Message Block) conocido como CIFS.

En general hay dos acercamientos para el diseño e implementación de un DFS: 1) basado en bloques y basado en objetos.

Los DFS basados en bloques generalmente garantizan 2 aspectos: 1) la unidad de escritura y lectura es a nivel de bloque, y los bloques pueden ser distribuidos en diferentes nodos, la idea es que los bloques de un archivo estén distribuidos en un conjunto de nodos. 2) el sistema operativo cliente de un DFS garantiza transparencia en el sentido de que la API ofrecida desde el SO es igual para acceder archivos locales que remotos, porque el DFS se integra con el sistema de gestión de archivos del sistema operativo (ej: NFS, AFS, SMB, etc).

Los DFS basados en objetos (object storage, ej: AWS S3), los dos aspectos anteriores se manejan así: 1) la unidad de distribución es a nivel de archivo y no de bloque, es decir, y se garantiza que se lee o escribe un archivo como un todo y no a nivel de bloque. No está diseñado como un sistema de acceso aleatorio al archivo sino secuencial. No soporta la operación de actualización parcial del archivo, sino que se debe reemplazar todo el archivo. Son sistemas distribuidos de archivos principalmente diseñado para un enfoque WORM (Read-Once-Read-Many). Típicamente estos DFS soporta altos niveles de escalabilidad, redundancia y rendimiento. Si bien desde el cliente tiene una visión de archivo completo, en el sistema de backend podría tener (y normalmente lo hay) un mecanismo de particionamiento del archivo por bloques u otro criterio para mejorar la escalabilidad, tolerancia a fallos y rendimiento. 2) el sistema operativo local del cliente NO integra directamente la gestión de este DFS y en vez de ello se cuenta con un SDK o API para las diferentes primitivas de la gestión de archivos y normalmente tienen su propio CLI.

Referencias:

- GFS:
  - https://es.wikipedia.org/wiki/Google_File_System
  - The Google File System - https://g.co/kgs/XzwmU76
- HDFS
  - https://es.wikipedia.org/wiki/Hadoop_Distributed_File_System
  - The Hadoop Distributed File System – link

Desarrollo del proyecto

1. Definición del Servicio

Cada grupo de estudiantes debe realizar la definición formal de este proyecto utilizando La Descripción del Servicio del enunciado, que permita hacer más explícito el entendimiento del mismo y los requerimientos funcionales y no funcionales que cada grupo tendrá.

Cada grupo de trabajo, se le asignará la opción 1 o la opción 2 para la implementación del servicio.

Ejemplo:

En este proyecto se construirá como **DFS por bloques,** siguiendo una arquitectura tipo HDFS:

- Un **NameNode** central que mantiene la **tabla de metadatos** (ubicación de bloques por archivo).
- Múltiples **DataNodes** que almacenan los bloques.
- Clientes que interactúan con el DFS a través de una **CLI (interfaz de línea de comandos)** o API.

Operaciones básicas:

- **put <archivo>**: dividir un archivo en bloques y distribuirlos en DataNodes
- **get <archivo>**: recuperar un archivo reconstruyéndolo a partir de los bloques
- **ls, rm, mkdir, rmdir**: comandos básicos de gestión del file system
- Cada cliente gestiona **solo sus propios archivos** mediante una autenticación básica (user/pass).
- Además debe contar con APIs – operaciones basica de lectura / escritura de archivos (bloques) desde clientes.

2. Arquitectura del Sistema

- **Definir el tipo de arquitectura de acuerdo a la opción asignada por el profesor:** C/S, P2P o alguna variante o integración de las mismas

Ejemplo:

- **Tipo de arquitectura:** Maestro – Trabajador (Master–Workers) o P2P no estructurado con servidor replicado central de metadatos.

3. Comunicaciones de red: el proyecto debe correr o ejecutar sobre Internet (p. ej., nodos comunicándose por REST, gRPC o sockets TCP).

Componentes:

- **Cliente:** CLI/API para subir, bajar y gestionar archivos.
- **ControlNode:** servidor central de metadatos (ubicación de bloques y tamaño de archivos).
- **DataNode:** servidores que almacenan bloques.

Protocolos de Comunicación

- Definir cómo se comunican:
  - Cliente ↔ ControlNode
    Cliente ↔ DataNode
  - ControlNode <-> ControlNode
  - ControlNode <-> DataNode
  - DataNode <-> DataNode
- Ejemplos:
  - REST sobre HTTP
  - gRPC para comunicaciones más eficientes
  - WebSockets (si se requiere bidireccionalidad)
  - MOMs si requiere comunicaciones asincrónicas (ej: apache kafka, rabbitmq, etc)

5. Infraestructura y Entorno

- Usar AWS Academy o GCP académico con máquinas virtuales (IaaS)
- Cada nodo debe ejecutarse nativamente o en un contenedor Docker
- Los contenedores pueden estar en diferentes máquinas (uso de puertos expuestos vía Internet o red local)
- Cada nodo debe exponer una API

Entregables

1. **Informe técnico (PDF o Word):**
   - Objetivo y Marco teórico breve
   - Descripción del servicio y problema abordado
   - Arquitectura del sistema y diagramas
   - Especificación de protocolos y APIs
   - Algoritmos de particionamiento y distribución
   - Descripción del entorno de ejecución nativo o en Docker
   - Pruebas y Análisis de resultados
2. **Código fuente** en repositorio GitHub (bien documentado)
3. **Video demostración (10-15 min):**
   - Explicación del sistema
   - Ejecución en vivo o simulada del procesamiento distribuido

Criterios de Evaluación

Criterio	Ponderación	Aspectos Evaluados
1. Definición del servicio y Diseño arquitectónico	**15%**	- … - …
2. Especificaciones de comunicaciones	**15%**	- … - …
3. diseño para escalabilidad	**15%**	- … - …
4. Alta Disponilidad	**15%**	- … - …
5. Replicación y consistencia de datos	**15%**	- … - …
5. Seguridad	**15%**	- … - …
6. Video e informe final	**10%**	- Video (10–15 min) claro mostrando la ejecución del sistema - Informe técnico completo (objetivos, arquitectura, implementación, pruebas, resultados) - Repositorio GitHub bien documentado y reproducible

Recursos Sugeridos

- **Sistema operativo Linux**
- **Docker:** para simular clúster local distribuido
- **FastAPI / Flask:** para APIs de comunicación entre nodos
- **gRPC:** para alternativas eficientes a HTTP REST
- **Apache kafka o rabbitmq** para comunicaciones asincrónicas
- **Bases de datos,** para almacenamiento temporal